Sensor placement: data wrangling using DSL allocation layer

Dusty Gannon

2026-09-21

Table of contents

1 Loading the raw data **1**

2 Computing proportion of each watershed allocated to the triad levels **4**

3 Terrain covariates **13**

4 Creating the candidate site data **13**

5 Saving data **17**

This document prepares the inputs used by `sensor_placement.qmd` and the optimization scripts (`python/sensor_site_optim.py`, `python/biodiv_site_optim.py`): it starts from an allocations layer (`ESRF_Allocations_20260825.gpkg`) sent by Kaitlyn Athearn on 25 August 2026, builds a grid of candidate sensor locations with their environmental covariates, and computes distance to the nearest road. **This document must be rendered before `sensor_placement.qmd`**, since the latter reads the files written here rather than re-deriving them.

1 Loading the raw data

I start from the `ESRF_Allocations_20260825` layer stored in `ESRF_Allocations_20260825.gpkg` provided by K. Athearn and use the stand and watershed attributes therein.

> **!** Important
>
> The most up-to-date allocation information is in the `ops_allocation` field of this layer and *not* the `hcp_allocation` field, which reflects historical allocations. This version of the document uses the `ops_allocation` field.

```
import pandas as pd
import numpy as np
import geopandas as gpd
from skimpy import clean_columns
from pyprojroot.here import here

f = here('Data/esrf_alloc/ESRF_Allocations_20260825.gpkg')
bio_f = here(
    'Data/esrf_alloc/biodiv_sites/'
    'ESRF 2023 Biodiversity Surveys_AdjustedSiteCentersInverts.shp'
)
roads_f = here('Data/esrf_alloc/ESRF_roads_ODF.gpkg')

# load data
esrf = gpd.read_file(f, layer='ESRF_Allocations_20260825')
bio_sites = gpd.read_file(bio_f)
roads = gpd.read_file(roads_f)

esrf = clean_columns(esrf)
bio_sites = clean_columns(bio_sites)
roads = clean_columns(roads)

# convert ops_allocation values to snake_case
esrf['ops_allocation'] = (
    esrf['ops_allocation']
    .str.replace(r'[^0-9a-zA-Z]+', '_', regex=True)
    .str.lower()
)

# convert hcp allocation to snake case
esrf['hcp_allocation'] = (
    esrf['hcp_allocation']
    .str.replace(r'[^0-9a-zA-Z]+', '_', regex=True)
    .str.lower()
)
```

```
# convert treatments to snake case
esrf['hcp_ws_assignment'] = (
    esrf['hcp_ws_assignment']
    .str.replace(r'[^0-9a-zA-Z]+', '_', regex=True)
    .str.lower()
)

# remove riparian buffers from consideration
esrf = esrf.loc[
    ~esrf['ops_allocation'].str.contains(r'_rca', regex=False),
    :
]
```

> **!** Important
>
> Note that the CBMA-CRW polygons include what was allocated as RCA. I am not sure if this was by design or if the RCA polygons need to be clipped from the CBMA-CRW polygons.

Table 1: Summary of acres allocated to each management type. Some types may be polygon slivers left over from previous data layers and need to be cleaned up.

Allocation code	Total acreage	Description
cbma_crw	10013	Carbon and biodiversity acreas (DSL FMP 2024)
crw_reserve_no_thin	16443	CRW Reserve
crw_reserve_thin	6829	CRW Reserve with early thinning
extensive	9516	Extensive
flexible_extensive	11	Flexible
intensive	9740	Intensive harvest
mrw_flexible	5941	Flexible
mrw_reserve	10293	MRW Reserve
volume_replacement	12	Volume replacement

2 Computing proportion of each watershed allocated to the triad levels

The new allocation layer does not designate triad treatments to watersheds. With the changes to allocations, watersheds that were assigned to triad treatments, particularly those assigned to the Triad-E treatment, have much different proportions of each stand treatment type (Reserve,

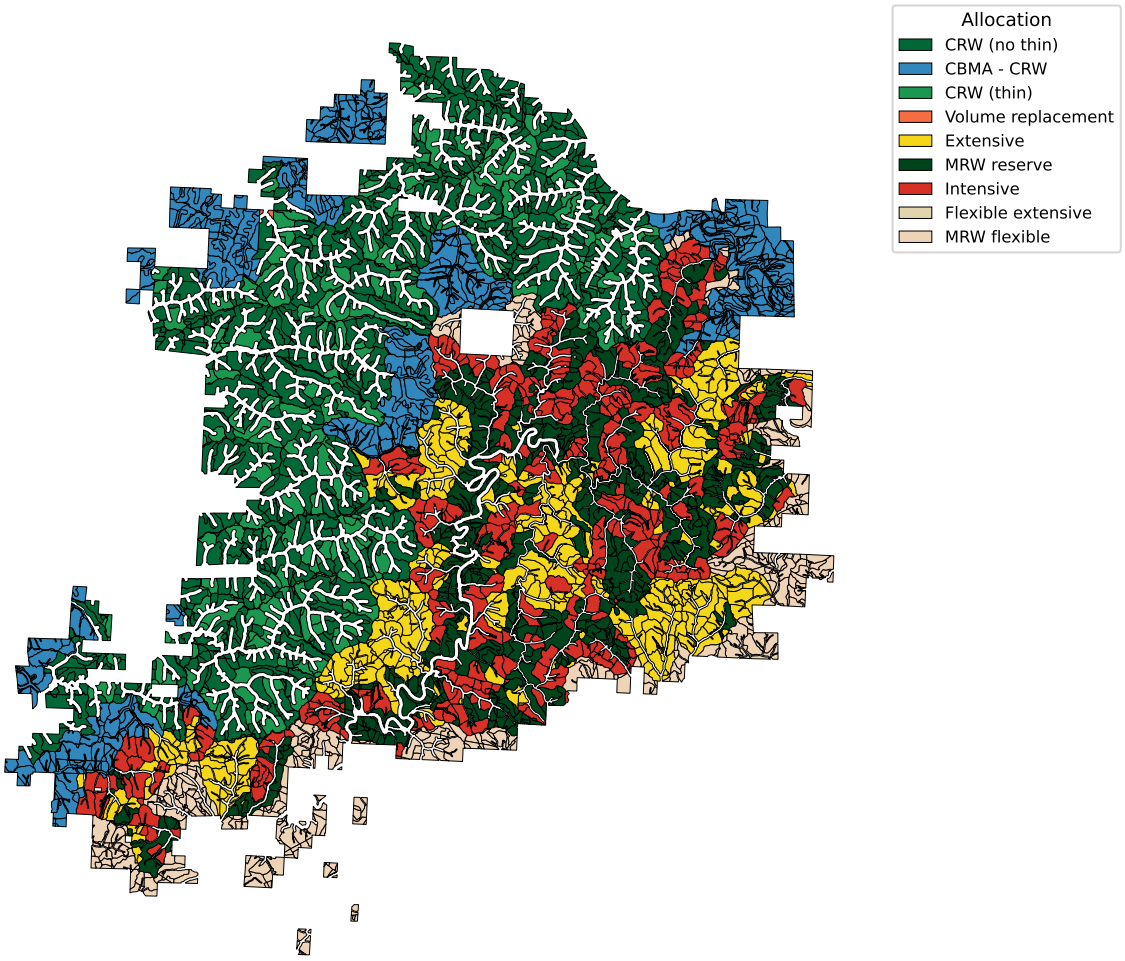


Figure 1: Map of stand allocations based on DSL (`ops_allocation`) allocations as of August, 2026. Riparian conservation areas have been masked, except for inside the CBMA-CRW watersheds, where additional geoprocessing is necessary.

Extensive, or Intensive) now. Table 2 lists the changes made to each watershed while Figure 2 shows how the watersheds fall in the ternary plot, colored by their original triad assignments.

Table 2: List of changes made to each watershed from the HCP allocations to the most recent DSL allocations. Note that CBMA-CRW allocations have not removed the stream buffers (RCA) from the polygons, so acreage may not be correct.

hcp_ws	None	crw_rca_added	crw_reserve_no_cbma_crw	crw_reserve_thin_cbma_crw	flexible_extensive_cbma_crw	intensive_cbma_crw	crw_flexible_rca_cbma_crw	crw_reserve_rca_cbma_crw	volunteer_replacement_crw
1	41	0	0	0	0	0	471	72	0	0
2	114	0	0	0	0	0	848	156	0	0
3	0	0	0	0	0	0	79	14	0	0
4	989	0	0	0	0	0	0	22	56	0
5	0	0	0	0	0	0	39	0	0	0
6	303	0	0	0	0	0	0	22	171	0
7	106	0	0	0	0	0	0	0	0	0
8	439	0	0	0	0	0	0	0	12	0
9	39	0	0	0	0	0	0	0	0	0
10	614	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	217	5	0	0
12	404	0	0	0	0	0	0	0	0	0
13	1201	0	2	0	0	0	0	0	0	0
14	1367	0	0	0	0	0	0	0	0	0
15	822	0	0	0	0	0	0	0	0	0
16	144	0	0	0	0	0	0	0	0	0
17	15	0	0	0	0	0	171	11	0	0
18	83	0	0	0	0	0	0	0	0	0
19	1304	0	0	0	0	0	0	0	0	0
20	367	0	0	0	0	0	0	0	0	0
21	552	0	0	0	0	0	0	0	0	0
22	432	0	0	0	0	0	0	0	0	0
23	388	0	0	0	0	0	0	0	0	0
25	383	0	0	0	0	0	0	0	0	0
26	0	0	0	0	439	0	165	0	96	197	0
27	0	0	0	0	0	0	0	0	0	0	41
28	104	55	58	0	0	0	0	0	63	0	268
29	131	0	0	0	0	0	0	0	0	0	2
30	0	77	95	40	0	0	0	0	64	0	279
31	0	38	74	37	0	0	0	0	0	0	32
32	1165	62	37	133	0	0	0	0	26	0	82
33	464	0	0	0	0	0	0	0	0	0	0

Table 2: List of changes made to each watershed from the HCP allocations to the most recent DSL allocations. Note that CBMA-CRW allocations have not removed the stream buffers (RCA) from the polygons, so acreage may not be correct.

hcp_ws	No_add	crw_rca_added	crw_reserve_no_cbma_crw	crw_reserve_thin_cbma_crw	flexible_extensive_cbma_crw	intensive_cbma_crw	crw_flexible_rca_cbma_crw	crw_reserve_rca_cbma_crw	volunteer_replacement_crw
34	413	0	0	0	0	0	0	0	0	0
35	0	0	0	0	0	0	0	0	0	13
36	0	0	0	0	0	0	0	0	0	11
37	34	18	47	0	0	0	0	8	0	57
38	0	0	0	0	0	0	0	0	0	24
39	216	0	0	0	0	175	0	58	0	0
40	615	14	66	0	0	0	0	33	0	102
41	0	0	0	0	0	0	0	0	0	19
42	235	0	0	0	0	0	0	0	0	0
43	698	0	0	0	0	0	0	0	0	0
44	520	0	0	0	0	0	0	0	0	0
45	110	0	0	0	0	0	0	0	0	0
46	319	0	0	0	0	460	0	0	179	34	0
47	668	0	0	0	0	0	0	0	0	0	0
48	729	0	0	0	0	0	0	0	0	0	0
49	129	0	0	0	0	0	0	0	0	0	0
50	1092	0	0	0	0	0	0	0	0	0	0
51	1065	0	0	0	0	0	0	0	0	0	0
52	566	0	0	0	322	0	1	0	79	192	0
53	496	0	0	0	0	0	0	0	0	0	0
54	233	0	0	0	0	0	0	0	0	0	0
55	584	0	0	0	0	0	0	0	0	0	0
56	685	0	0	0	0	0	0	0	0	0	0
57	88	0	0	0	0	0	0	41	2	0	0
58	75	0	0	0	0	0	0	33	9	0	0
59	69	14	5	0	0	0	0	214	11	0	0
60	310	0	0	0	0	0	0	0	0	0	0
61	866	0	0	0	0	0	0	0	0	0	0
62	142	0	0	0	0	0	0	0	0	0	0
63	1215	0	0	0	0	0	0	0	0	0	0
64	795	0	0	0	0	0	0	0	0	0	0
65	238	0	0	0	0	0	0	0	29	143	0
66	340	0	0	0	140	0	0	0	51	201	0
67	796	7	0	0	0	173	0	332	38	0	0
68	828	0	0	0	30	0	6	0	60	389	0

Table 2: List of changes made to each watershed from the HCP allocations to the most recent DSL allocations. Note that CBMA-CRW allocations have not removed the stream buffers (RCA) from the polygons, so acreage may not be correct.

hcp_watershed	No_Action	crw_rca_added_to_cbma_crw	crw_reserve_no_stream_buffers_cbma_crw	crw_reserve_thinning_cbma_crw	flexible_extensive_cbma_crw	intensive_cbma_crw	crw_flexible_rca_cbma_crw	crw_reserve_rca_cbma_crw	volunteer_replacement_cbma_crw
69	335	0	0	0	0	0	0	0	0	0	0
70	173	0	0	0	0	0	0	0	0	0	0
71	443	0	0	0	0	0	0	0	0	0	0
72	588	0	0	0	0	0	0	0	0	0	0
73	792	0	0	0	0	0	0	0	0	0	0
74	363	0	0	0	0	0	0	0	0	0	0
75	459	0	0	0	0	0	0	0	0	0	0
76	658	0	0	0	0	0	0	0	0	0	0
77	277	0	0	0	0	0	0	0	0	0	0
78	216	0	0	0	0	0	0	0	0	0	0
79	77	0	0	0	0	0	0	0	0	0	0
81	422	0	0	0	0	0	0	0	0	0	0
82	472	0	0	0	0	0	0	0	0	0	0
83	177	0	0	0	0	0	0	0	0	0	0
84	186	0	0	0	0	0	0	0	0	0	0
85	416	0	0	0	0	0	0	0	0	0	0
86	195	0	0	0	0	0	0	0	0	0	0
87	50	0	0	0	0	0	0	0	0	0	0
88	22	0	0	0	0	0	0	0	0	0	0
89	39	0	0	0	0	0	0	0	0	0	0
90	907	0	0	0	0	0	0	0	0	0	0
91	646	0	0	0	0	0	0	0	0	0	0
92	46	0	0	0	0	0	0	0	0	0	0
93	1251	0	0	0	0	0	0	0	0	0	0
94	1208	0	0	0	0	0	0	0	0	0	0
95	1345	0	0	0	0	0	0	0	0	0	0
96	739	0	0	0	0	0	0	0	0	0	0
97	1037	0	0	0	0	0	0	0	0	0	0
98	1240	0	0	0	0	0	0	0	0	0	0
99	1508	0	0	0	0	0	0	0	0	0	0
100	848	0	0	0	0	0	0	0	0	0	0
101	366	0	0	0	0	0	0	0	0	0	0
103	132	0	0	0	0	0	0	177	24	0	0
104	314	0	0	0	0	0	0	0	0	0	0
105	1250	0	0	0	0	0	0	0	0	0	0

Table 2: List of changes made to each watershed from the HCP allocations to the most recent DSL allocations. Note that CBMA-CRW allocations have not removed the stream buffers (RCA) from the polygons, so acreage may not be correct.

hcp_wsno	None	crw_rca_added	crw_reserve_no_change	crw_reserve_thin_crwcbma_crw	crw_reserve_thin_crwcbma_crw	flexible_extensive_crwcbma_crw	flexible_extensive_crwcbma_crw	intensive_crwcbma_crw	intensive_crwcbma_crw	crw_flexible_rca_crwcbma_crw	crw_flexible_rca_crwcbma_crw	crw_resource_crwcbma_crw	volunteer_replacement_crw
106	1118	0	0	0	0	0	0	0	0	0	0	0	0
107	407	0	0	0	0	0	0	0	0	0	0	0	0
108	366	0	0	0	0	0	0	0	0	0	0	0	0
109	436	0	0	0	0	0	0	0	0	0	13	0	0
110	889	0	0	0	0	0	0	0	0	0	0	0	0
111	1135	0	0	0	0	0	0	0	0	0	0	0	0
112	729	0	0	0	0	0	0	0	0	0	0	0	0
113	989	0	0	0	0	0	0	0	0	0	0	0	0
114	1033	0	0	0	0	0	0	0	0	0	0	0	0
115	368	0	0	0	0	0	0	0	0	0	0	0	0
116	53	0	0	0	0	0	0	0	0	0	0	0	0
118	138	0	0	0	0	0	0	0	0	0	0	0	0
119	1189	0	0	0	0	0	0	0	0	0	0	0	0
120	262	0	0	0	0	0	0	0	0	0	0	0	0
121	395	0	0	0	0	0	0	0	0	0	0	0	0
122	304	0	0	0	0	0	0	0	0	0	0	0	0
123	55	0	0	0	0	0	0	0	0	0	0	0	0
124	154	0	0	0	0	0	0	0	0	0	0	0	0
140	0	0	0	0	0	0	0	0	4	0	0	0	0
212	120	0	0	0	0	0	0	0	0	0	0	0	0
215	24	0	0	0	0	0	0	0	0	0	0	0	0
216	29	0	0	0	0	0	0	0	0	0	0	0	0
219	18	0	0	0	0	0	0	0	0	0	0	0	0
221	27	0	0	0	0	0	0	0	0	0	0	0	0
222	264	0	0	0	0	0	0	0	0	0	0	0	0
223	277	0	0	0	0	0	0	0	0	0	0	0	0
224	31	0	0	0	0	0	0	0	0	0	0	0	0
244	33	0	0	0	0	0	0	0	0	0	0	0	0
265	204	0	0	0	0	0	0	0	0	0	0	0	0
268	3	0	0	0	0	0	0	299	24	0	0	0	0
275	17	0	0	0	0	0	0	0	0	0	0	0	0
276	53	0	0	0	0	0	0	0	0	0	0	0	0
277	207	0	0	0	0	0	0	0	0	0	0	0	0
279	19	0	0	0	0	0	0	0	0	0	0	0	0
280	408	0	0	0	0	0	0	0	0	0	0	0	0

Table 2: List of changes made to each watershed from the HCP allocations to the most recent DSL allocations. Note that CBMA-CRW allocations have not removed the stream buffers (RCA) from the polygons, so acreage may not be correct.

hcp_ws	None	crw_rca_added	crw_reserve_no_thin_cbma_crw	crw_reserve_thin_cbma_crw	flexible_extensive_cbma_crw	intensive_cbma_crw	crw_flexible_rca_cbma_crw	crw_reserve_rca_cbma_crw	volume-replacement-cbma_crw
281	14	0	0	0	0	0	0	0	0	0	0
285	58	0	0	0	0	0	0	0	0	0	0
286	12	0	0	0	0	0	0	0	0	0	0
290	3	0	0	0	0	0	0	0	0	0	0
291	59	0	0	0	0	0	0	0	0	0	0
292	110	0	0	0	0	0	0	0	0	0	0
293	4	0	0	0	0	0	0	0	0	0	0
315	286	0	0	0	0	0	0	0	0	0	0
316	46	0	0	0	0	0	0	0	0	0	0
360	16	0	0	0	0	0	0	24	0	0	0
361	84	0	0	0	0	0	0	0	0	0	0
999	0	0	0	0	409	0	0	0	82	288	0

The important take-aways from Table 2 and Figure 2 are that the Triad-E watersheds have been pushed towards higher proportions of reserve than the original design. This indicates that a regression-type approach, using mixture models may be better than considering the watersheds as belonging to the original treatment designations. I therefore use the proportions of area allocated to each stand treatment type to redesignate the watershed treatments. For example, watershed 26 must be reassigned from Triad-E to CRW (reserve).

> **!** Important
>
> I use the watershed treatment types (CRW reserve, Intensive, Triad-I, Triad-E, and Extensive) as a starting point to ensure the initial placement of sensors spans a good range of the triad experiment, but use a **regression approach** with continuous measures of proportions of each watershed type in defining the objective function for the optimization scheme. The watershed treatment assignments below are therefore just a construct to aid in visualization and initialization.

To reassign watersheds to original treatment constructs, I define a band around the Cox direction for the Extensive treatment Figure 3. Treatments are then designated by the proportion of Extensive, with less than 5% as Intensive, between 5% and 40% as Triad-I, between 40% and 80% as Triad-E, and more than 80% as Extensive. Those with more than 95% reserve are designated as control/reserve watersheds. Watersheds falling outside the band around the Cox direction are left unassigned but elligible for sensor placement (colored purple in Figure 3).

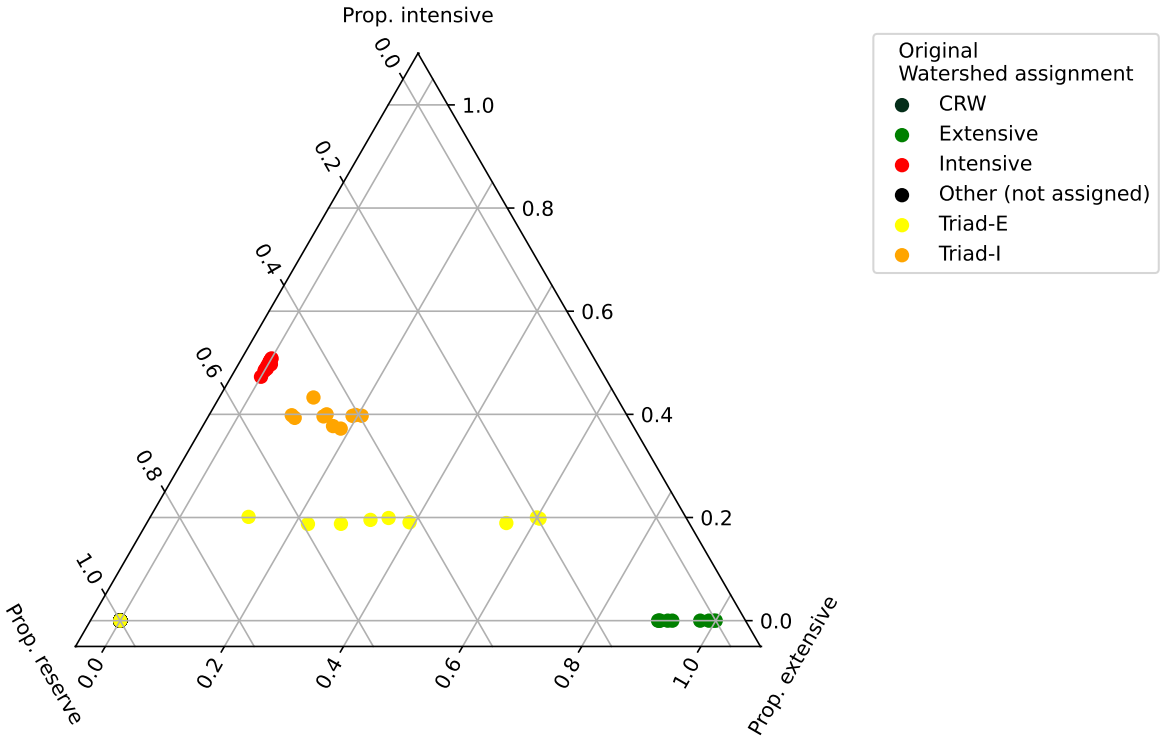


Figure 2: Ternary plot of watersheds and proportion of each allocated to one of three management types. Note that the grey points represent other watersheds not assigned to a treatment in the original design.

```
# redesignate the watersheds based on where they fall in the ternary
# plot above. Watersheds with no original assignment, or assigned
# crw_partial, keep their original hcp_ws_assignment value. A watershed
# is only assigned one of the triad classes if it falls within a band
# around the reserve == intensive diagonal; its sub-class is then
# determined by prop_extensive. This mirrors classify_triad() below,
# used to trace the same boundaries over the full simplex.
eligible = (
    triad['hcp_ws_assignment'].notna()
    & (triad['hcp_ws_assignment'] != 'crw_partial')
)
in_band = (triad['prop_reserve'] - triad['prop_intensive']).abs() <= 0.2

triad['triad_trt2'] = np.select(
    condlist=[
        eligible & (triad['prop_reserve'] > 0.95),
        eligible & in_band & (triad['prop_extensive'] < 0.05),
        eligible & in_band & (triad['prop_extensive'] >= 0.05) & (triad['prop_extensive'] < 0
        eligible & in_band & (triad['prop_extensive'] >= 0.4) & (triad['prop_extensive'] < 0
        eligible & in_band & (triad['prop_extensive'] >= 0.8),
    ],
    choicelist=['crw_full', 'intensive', 'triad_i', 'triad_e', 'extensive'],
    default=np.where(eligible, 'none', triad['hcp_ws_assignment'])
)
```

The stand-level allocations determine the columns of $\mathbf{X}$ (the proportions of reserve, intensive, and extensive), but we want to make sure that we don't accidentally confound environmental conditions with the triad experiment when choosing the sensor locations. To do this, we add a DEM raster to get elevation, slope, and aspect. We also use the stand age as another control variable to help determine placement.

3 Terrain covariates

Elevation, slope, and aspect are derived from a 30 m DEM (USGS 3DEP, via `py3dep`), smoothed and downsampled to a 120 m grid, in `python/get_course_dem.py`. That script writes a 4-band raster (elevation, slope, aspect_x, aspect_y) to `Data/esrf_alloc/dem_esrf_120_terrain.tif`, which I load here.

Using the centroids of each 120 m grid cell in the landscape, we extract the geospatial data to a spatial points data frame. This provides a list of candidate locations at which to locate sensor nodes.

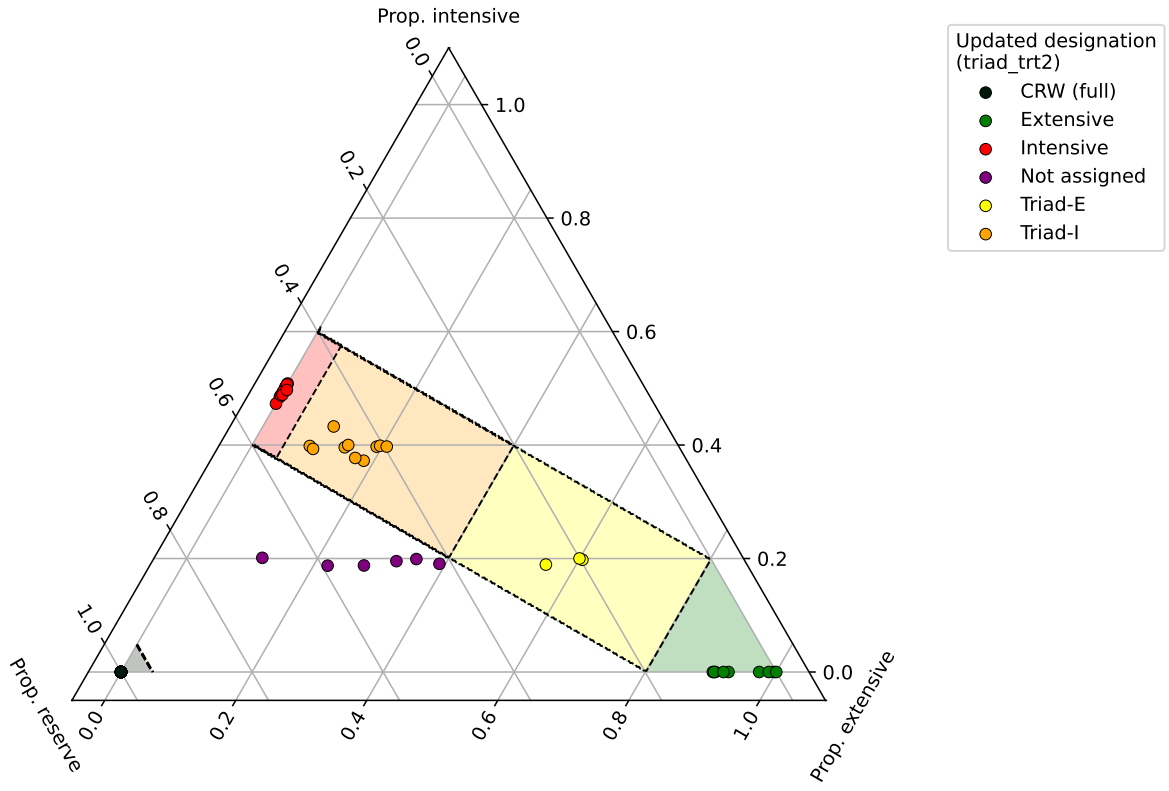


Figure 3: Ternary plot of watersheds using the updated `triad_trt2` designations, with the decision boundaries from the redesignation rule overlaid (dashed lines). Shaded regions show where each rule applies across the full simplex; points show the actual watersheds.

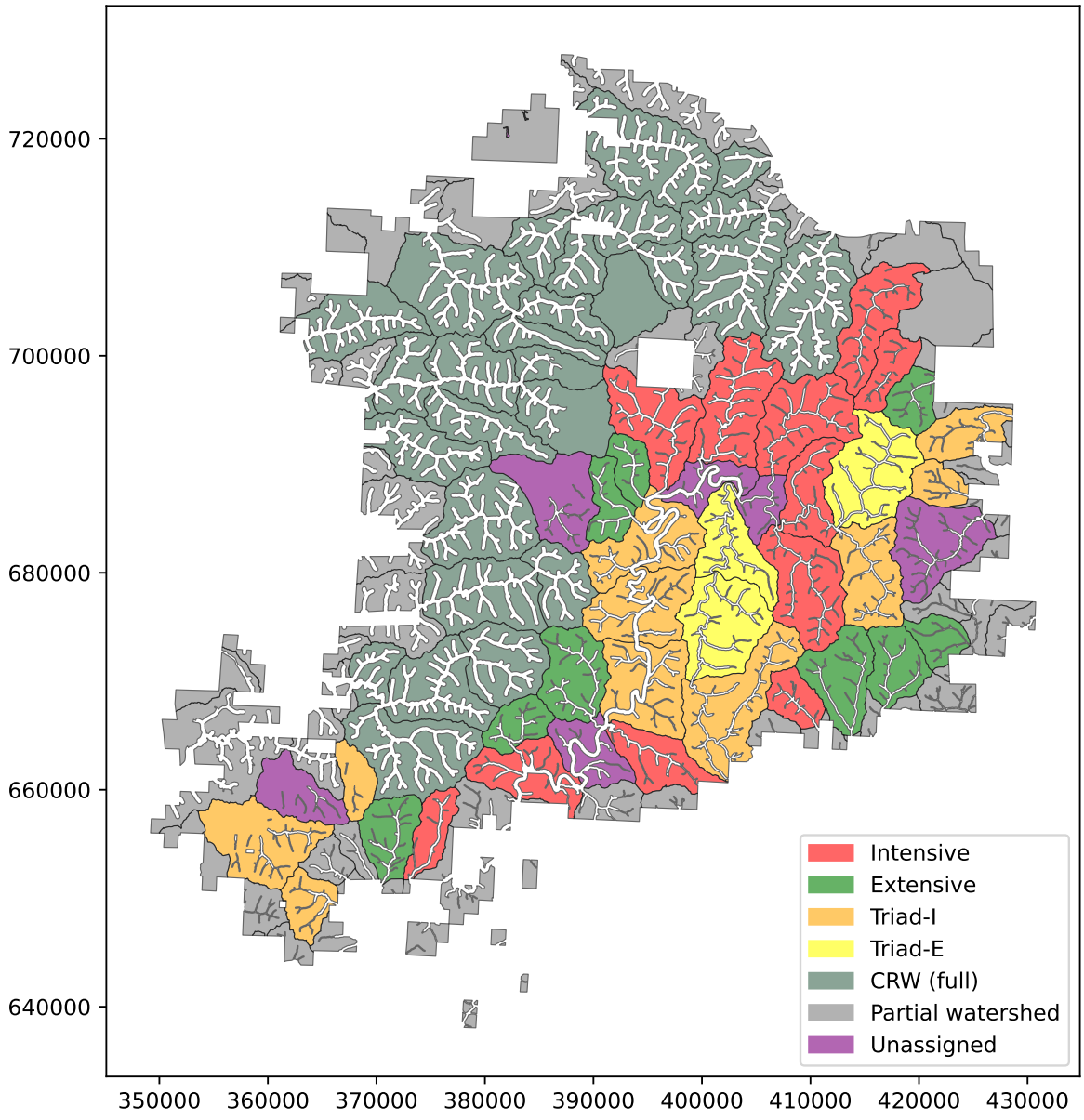


Figure 4: Visual of the spatial arrangement of guiding "treatments". The watersheds that fell outside of the original Triad design after adjustments are now labeled as "unassigned". These will still be considered during sensor placement, but not used in the strict backbone of the original placements.

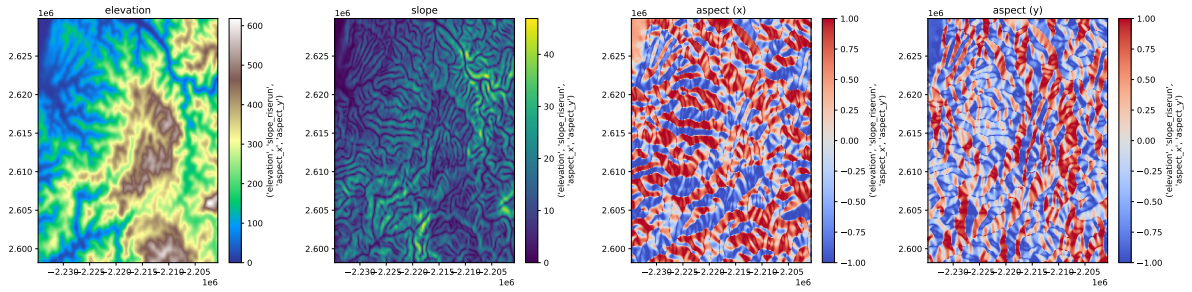


Figure 5: Terrain covariates derived from the 120 m DEM: elevation, slope, and aspect (decomposed into x/y components).

4 Creating the candidate site data

```
from shapely.geometry import Point

esrf_proj = esrf.to_crs(dem_120_terrain.rio.crs)
# simplified stand-level allocation, for stratifying site placement by
# stand allocation within a watershed treatment (see ALLOC_SIMPLE_MAP above)
esrf_proj['alloc_simple'] = esrf_proj['ops_allocation'].replace(ALLOC_SIMPLE_MAP)
# attach per-watershed treatment proportions so candidate_locs carries them too
esrf_proj = esrf_proj.merge(
    triad[['hcp_wtr_shd_id', 'prop_intensive', 'prop_extensive', 'prop_reserve', 'triad_trt2
    on='hcp_wtr_shd_id',
    how='left'
)
bio_proj = bio_sites.to_crs(dem_120_terrain.rio.crs)
roads_proj = roads.to_crs(dem_120_terrain.rio.crs)
eus_proj = eus.to_crs(dem_120_terrain.rio.crs)

# Build spatial points from the terrain raster's cell centers
xs = dem_120_terrain.x.values
ys = dem_120_terrain.y.values
elev_vals = elevation.values
slope_vals = slope.values
aspect_x_vals = aspect_x.values
aspect_y_vals = aspect_y.values
xx, yy = np.meshgrid(xs, ys)
x_ind, y_ind = np.meshgrid(
    range(len(xs)),
    range(len(ys))
```

```
)

nodata = dem_120_terrain.rio.nodata
mask = (elev_vals != nodata) if nodata is not None else np.ones(elev_vals.shape, dtype=bool)

candidate_locs = gpd.GeoDataFrame(
    {
        'elevation': elev_vals[mask],
        'slope': slope_vals[mask],
        'aspect_x': aspect_x_vals[mask],
        'aspect_y': aspect_y_vals[mask],
        'index_x': x_ind[mask],
        'index_y': y_ind[mask]
    },
    geometry=gpd.points_from_xy(xx[mask], yy[mask]),
    crs=dem_120_terrain.rio.crs
)

# Spatial join: attach triad polygon attributes to each point in the grid
candidate_locs = gpd.sjoin(
    candidate_locs,
    esrf_proj[[
        'geometry', 'age_2020', 'triad_trt2', 'ops_allocation',
        'alloc_simple', 'hcp_wtr_shd_id', 'prop_intensive', 'prop_extensive',
        'prop_reserve'
    ]],
    how='left',
    predicate='within'
)
# drop the 'index_right column'
candidate_locs.drop('index_right', axis=1, inplace=True)

# conduct a spatial join with the biodiversity sites
## first grab the columns we actually want
bio_proj_cols = ['station_id', 'geometry']
bio_proj_sa = gpd.sjoin(
    bio_proj[bio_proj_cols],
    eus_proj[['hcp_wtr_shd_id', 'geometry']],
    how = 'inner',
    predicate = 'within'
)
bio_proj_sa.drop(['index_right', 'hcp_wtr_shd_id'], axis = 1, inplace = True)
```

```
# now find the nearest candidate to each biodiv site within
# the study area
bio_div_join = gpd.sjoin_nearest(
    bio_proj_sa,
    candidate_locs.reset_index(names = "cell_id")[['cell_id', 'geometry']],
    how = 'left'
)
bio_div_join.drop(['index_right'], axis = 1, inplace = True)
bio_div_join.rename({'geometry': 'geometry_bio'}, axis = 1, inplace = True)

# now join these back to identify which cells contain the the biodiv sites
candidate_locs = bio_div_join.join(candidate_locs, on='cell_id', how="right")

# fill the nas in the station_id
candidate_locs['station_id'] = candidate_locs['station_id'].fillna('none')
# only keep the grid cell locations (i.e., "snap" the biodiv monitoring sites to the grid cel
candidate_locs.drop(['geometry_bio'], axis = 1, inplace = True)
candidate_locs = candidate_locs.dropna()

# create filter mask
mask = np.array(np.repeat(1, candidate_locs.shape[0]), dtype = 'bool')
mask[candidate_locs['triad_trt2'].isna().values] = False
mask[
    (candidate_locs['triad_trt2'] == 'crw_partial')
    .values
] = False

# filter out the treatments we are not interested in
candidate_locs = candidate_locs[mask]
```

Next step is to compute, for each point, the distance to the nearest road based on the roads layer.

```
candidate_locs = candidate_locs.sjoin_nearest(
    roads_proj,
    'left',
    distance_col = "dist_to_road_m"
)
candidate_locs.drop('index_right', axis=1, inplace=True)
```

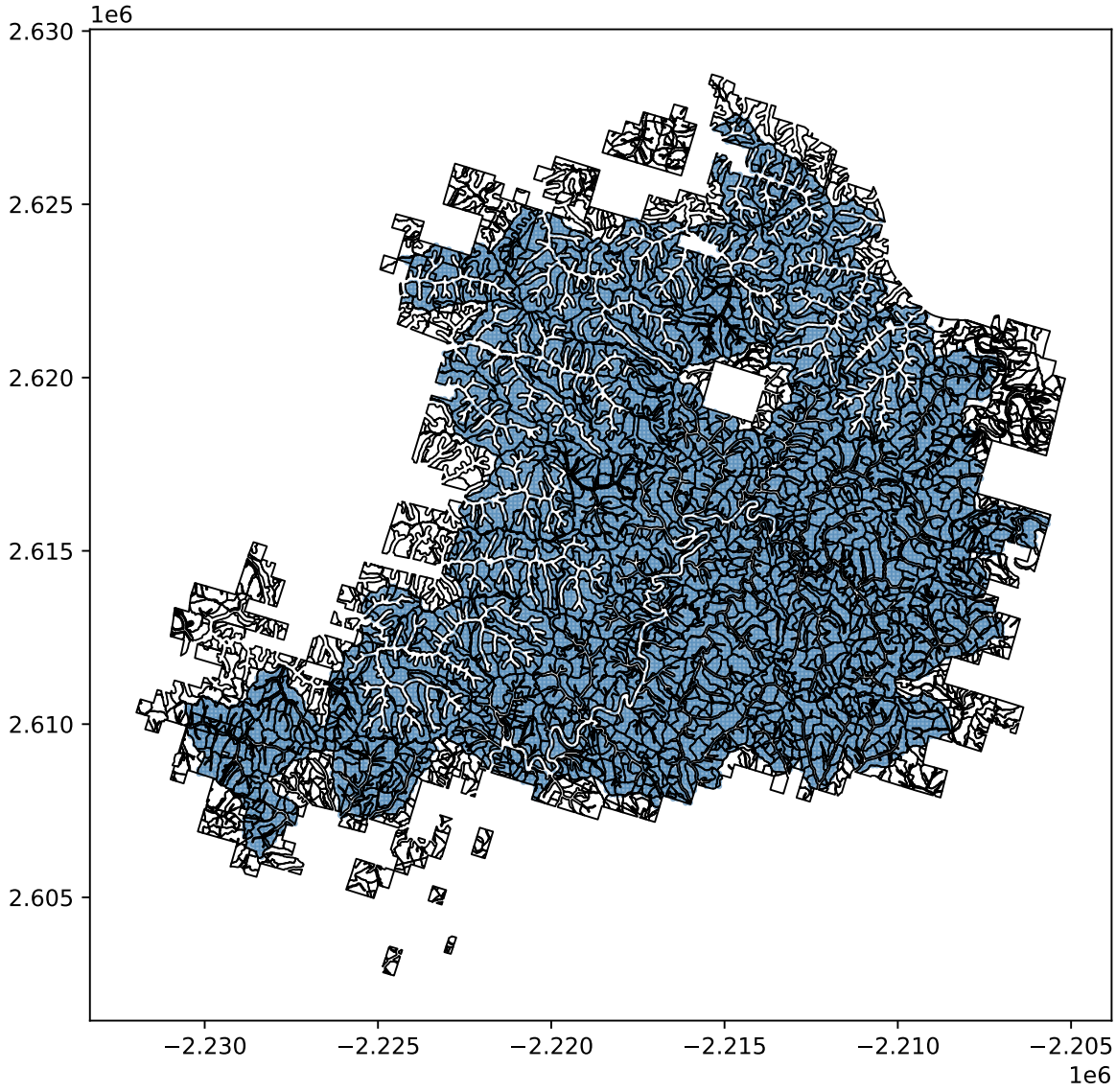


Figure 6: Candidate locations with the watershed boundaries overlaid. Note that the partial wartersheds do not get considered.

5 Saving data

I save all the derived data layers to a geopackage for use in the sensor placement analysis.

```
import os

if not os.path.isdir(here('Data/sensor_optim')):
    os.mkdir(here('Data/sensor_optim'))

# save eus_proj, since it is the same CRS as candidates
esrf.to_file(
    here('Data/sensor_optim/data_for_optim.gpkg'),
    driver = 'GPKG',
    layer = 'esrf_alloc'
)

eus.to_crs(esrf.crs).to_file(
    here('Data/sensor_optim/data_for_optim.gpkg'),
    driver='GPKG',
    layer='eus'
)

# add layer to sensor optim data file
# using same crs as eus
candidate_locs.to_crs(esrf.crs).to_file(
    here('Data/sensor_optim/data_for_optim.gpkg'),
    driver='GPKG',
    layer='candidate_locs'
)

# reprojected biodiversity sites, for use in result maps in the main report
bio_proj.to_crs(esrf.crs).to_file(
    here('Data/sensor_optim/data_for_optim.gpkg'),
    driver='GPKG',
    layer='bio_sites'
)
```